

Felipe William Galdino da Silva,
João Carlos G. Iannuzzi,
Reyner C. Silva Alegria

Relatório Técnico - Processamento Paralelo em Cluster Ubuntu

Itacoatiara - Am
2026

Felipe William Galdino da Silva,
João Carlos G. Iannuzzi,
Reyner C. Silva Alegria

Relatório Técnico - Processamento Paralelo em Cluster Ubuntu

Trabalho apresentado ao professor Antonio Alberto Sena dos Santos do Instituto de Ciências Exatas e Tecnologia da Universidade Federal do Amazonas, como atividade prática para a obtenção de nota na disciplina de Sistemas Distribuídos.

Itacoatiara - Am
2026

SUMÁRIO

1 INTRODUÇÃO

O presente trabalho tem como propósito a configuração e experimentação de um ambiente de computação distribuída, utilizando um *cluster* composto por quatro máquinas virtuais orquestradas sob o paradigma de processamento paralelo com MPI (*Message Passing Interface*). O problema central explorado é a estimativa do valor de π (Pi) por meio do Método de Monte Carlo — uma técnica probabilística que simula o lançamento de um bilhão de pontos aleatórios em um espaço bidimensional, utilizando a proporção dos pontos que caem dentro de um círculo inscrito em um quadrado para aproximar o valor da constante matemática. Embora conceitualmente simples, o volume massivo de iterações exigido torna a execução sequencial em uma única máquina computacionalmente custosa, evidenciando a necessidade de distribuição da carga de trabalho. Nesse contexto, a atividade propõe a divisão igualitária das iterações entre um nó mestre e três nós trabalhadores (*workers*), permitindo que o processamento ocorra em paralelo e, ao final, os resultados parciais sejam consolidados via *MPI Reduce*. O experimento visa não apenas verificar a aceleração (*speedup*) obtida com o uso do *cluster* em relação à execução serial, mas também aprofundar a compreensão sobre os três pilares fundamentais dos sistemas distribuídos: comunicação entre nós via rede e SSH, sincronização das tarefas paralelas através do MPI, e escalabilidade do sistema frente ao aumento do número de processos.

2 METODOLOGIA

A metodologia desta atividade foi estruturada em cinco fases sequenciais, cada uma construindo sobre a anterior para garantir um ambiente distribuído funcional e reproduzível. A seguir, cada fase é detalhada com suas justificativas técnicas.

2.1 Provisionamento das Máquinas Virtuais

O primeiro passo consistiu na criação de quatro máquinas virtuais (VMs) utilizando o hipervisor *VirtualBox*. A escolha por virtualização, em vez de máquinas físicas dedicadas, se justifica pela viabilidade prática do ambiente acadêmico: VMs permitem simular um *cluster* real em um único computador *host*, com custo zero de hardware adicional e total controle sobre a configuração de rede.

Foram criadas quatro VMs com as especificações descritas na Tabela ??:

Tabela 1 – Especificações das máquinas virtuais

Máquina	RAM	CPUs
master	1GB	2
node1	1 GB	2
node2	1GB	2
node3	1 GB	2

A alocação de apenas 1 GB de RAM e 2 CPUs por VM foi uma decisão deliberada para não sobrecarregar o computador *host*, já que quatro VMs rodam simultaneamente. O desempenho individual de cada nó é secundário neste experimento — o que se avalia é o ganho coletivo do *cluster*.

O sistema operacional escolhido foi o *Ubuntu Server 22.04 LTS*. A versão *Server* (sem interface gráfica) foi preferida à versão *Desktop* pois consome significativamente menos memória RAM e processamento, recursos escassos no cenário de múltiplas VMs simultâneas. A designação *LTS* (*Long Term Support*) garante estabilidade e suporte de atualizações de segurança por cinco anos, sendo a escolha padrão da indústria para servidores em produção.



Figura 1 – Demonstração do hipervisor mostrando as 4 VMs criadas.

2.2 Configuração de Rede e Identificação dos Nós

Para que as máquinas de um cluster possam se comunicar de forma confiável, cada VM foi configurada com dois adaptadores de rede no VirtualBox, cada um com uma função distinta e complementar.

O primeiro adaptador opera em modo **NAT** (*Network Address Translation*). Neste modo, o *VirtualBox* funciona como um roteador intermediário: a VM acessa a internet através do IP do computador hospedeiro, sem precisar de um IP público próprio. Este adaptador foi necessário exclusivamente para permitir o acesso à internet durante a instalação de pacotes via **apt** (compilador, MPI, SSH). Ele não é utilizado para a comunicação entre os nós do cluster.

O segundo adaptador opera em modo **Rede Interna** (*Internal Network*), com o nome de rede **cluster_net**. Este modo cria uma rede privada e isolada, visível *apenas* entre as VMs que compartilham o mesmo nome de rede — completamente separada da rede física do hospedeiro. É por esta interface que todo o tráfego do MPI e do SSH entre os nós circula. O isolamento garante que não haja interferência de outros dispositivos da rede física, tornando o ambiente do *cluster* controlado e previsível.

Com os adaptadores criados, foi necessário configurar um IP fixo na interface do Adaptador 2 (**enp0s8**) em cada VM. IPs fixos são indispensáveis em um *cluster*: se o endereço de um nó mudasse a cada reinicialização (como ocorre em DHCP), o `/etc/hosts` e os arquivos de configuração do MPI ficariam desatualizados, quebrando a comunicação.

No Ubuntu Server 22.04, a configuração de rede é gerenciada pelo *Netplan*. O arquivo de configuração foi editado no nó master e, posteriormente, adaptado em cada clone com o IP correspondente:

Listing 1 – Edição do arquivo de configuração do Netplan no master.

```
1 sudo nano /etc/netplan/00-installer-config.yaml
```

Listing 2 – Conteúdo do arquivo Netplan configurando NAT (DHCP) e IP fixo na rede interna.

```
1 network:
2   ethernets:
3     enp0s3:          # Adaptador 1 - NAT (internet via DHCP)
4       dhcp4: true
5     enp0s8:          # Adaptador 2 - Rede Interna (IP fixo)
6       addresses: [192.168.1.10/24]  # Ajustar por n
7   version: 2
```

A interface **enp0s3** corresponde ao Adaptador 1 (NAT) e recebe seu IP automaticamente via DHCP do VirtualBox. Já a **enp0s8** corresponde ao Adaptador 2 (Rede Interna) e recebe um IP fixo atribuído manualmente, diferente para cada nó. A notação `/24` define a máscara de sub-rede `255.255.255.0`, o que significa que todos

os nós do cluster (na faixa 192.168.1.x) pertencem à mesma rede e conseguem se comunicar diretamente.

Após editar o arquivo, as configurações foram aplicadas com:

Listing 3 – Aplicação das configurações de rede.

```
1 sudo netplan apply
```

Os IPs fixos atribuídos a cada nó na rede interna estão descritos na Tabela ??.

Máquina	Endereço IP
master	[192.168.1.10]
node1	[192.168.1.11]
node2	[192.168.1.12]
node3	[192.168.1.13]

Tabela 2 – Mapeamento de nomes e endereços IP do cluster.

Em seguida, o arquivo `/etc/hosts` foi editado em **todas as quatro VMs**. Este arquivo é um mecanismo de resolução de nomes local — ele instrui o sistema operacional a associar um nome amigável (como `master` ou `node1`) a um endereço IP, sem depender de um servidor DNS externo. Isso é essencial para o MPI, que localiza os nós do cluster pelos seus nomes de *host*.

Listing 4 – Edição do arquivo `/etc/hosts` em todos os nós.

```
1 sudo nano /etc/hosts
2
3 192.168.1.10 master
4 192.168.1.11 node1
5 192.168.1.12 node2
6 192.168.1.13 node3
```

2.3 Instalação das Dependências

Esta fase foi executada em todas as quatro VMs, pois cada nó do *cluster* precisa ser capaz de compilar e executar o código MPI de forma autônoma. Primeiramente, os repositórios do sistema foram atualizados para garantir a instalação das versões mais recentes dos pacotes:

Listing 5 – Atualização do sistema em todos os nós.

```
1 sudo apt update && sudo apt upgrade -y
```

Em seguida, foram instalados os três componentes essenciais:

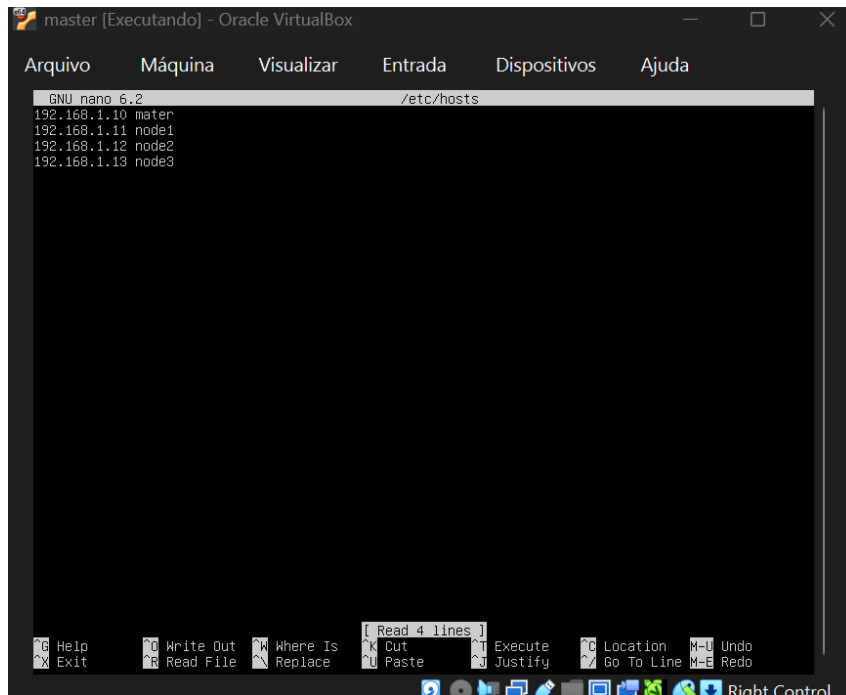


Figura 2 – Arquivo `/etc/hosts` configurado com os IPs e nomes do *cluster*.

Listing 6 – Instalação do compilador, servidor SSH e biblioteca MPI.

```
1 sudo apt install -y build-essential openssh-server openmpi-bin  
libopenmpi-dev nfs-kernel-server
```

Cada componente tem uma função específica e indispensável:

- **build-essential** — Instala o compilador GCC e ferramentas essenciais de compilação em C (como `make` e `libc`). É necessário porque o algoritmo de Monte Carlo foi escrito em linguagem C, que precisa ser compilada antes de ser executada.
- **openssh-server** — Instala e ativa o servidor SSH (*Secure Shell*) em cada nó. O MPI utiliza o SSH como canal de comunicação para iniciar processos remotamente nos nós *workers* a partir do master. Sem o SSH ativo em todos os nós, o master não consegue distribuir e iniciar a execução do programa.
- **openmpi-bin** — Instala os binários do OpenMPI, incluindo o comando `mpirun`, responsável por lançar o programa paralelamente em múltiplos processos e nós. O OpenMPI é uma implementação de código aberto, madura e amplamente utilizada do padrão MPI, sendo a escolha natural para ambientes acadêmicos e de pesquisa.

- **libopenmpi-dev** — Instala os cabeçalhos (*headers*) e bibliotecas de desenvolvimento do MPI. Esses arquivos são necessários em tempo de compilação: quando o compilador encontra a diretiva `#include <mpi.h>` no código-fonte, é aqui que ele busca as definições das funções MPI. Sem este pacote, a compilação falharia.
- **nfs-kernel-server** — Instala o servidor NFS (*Network File System*), o protocolo padrão do Linux para compartilhamento de diretórios entre máquinas em rede. No contexto deste cluster, o NFS permite que um diretório criado no nó master seja montado e acessado por todos os *workers* como se fosse uma pasta local. Isso elimina a necessidade de copiar manualmente o executável compilado para cada nó via `scp`: basta compilar uma vez no master e todos os nós enxergam o arquivo automaticamente pelo diretório compartilhado.

2.4 Configuração do SSH sem Senha

Esta é uma das fases mais críticas para o funcionamento do *cluster*. O `mpirun`, ao ser executado no master, inicia processos nos nós *workers* automaticamente, sem intervenção humana. Se o SSH solicitasse uma senha a cada conexão, o processo travaria.

A solução é a autenticação por par de chaves criptográficas RSA. Neste mecanismo, o master gera duas chaves matematicamente relacionadas: uma privada (guardada apenas no master) e uma pública (distribuída para todos os nós). Quando o master tenta se conectar a um nó, o nó verifica a assinatura com a chave pública — se corresponder à chave privada, a autenticação é aprovada sem necessidade de senha.

Passo 1 — Geração do par de chaves no master.

Listing 7 – Geração do par de chaves RSA no nó master.

```
1 ssh-keygen -t rsa -N "" -f ~/.ssh/id_rsa
```

O parâmetro `-t rsa` especifica o algoritmo RSA para geração das chaves. Ao ser solicitado um caminho e uma *passphrase*, pressiona-se **Enter** em todas as opções para aceitar os valores padrão e manter a chave sem senha adicional.

Passo 2 — Distribuição da chave pública para todos os nós.

Ainda no nó master, a chave pública foi distribuída para todos os nós do cluster — inclusive para o próprio master — por meio de um laço `for` executado

```
Last login: Tue Mar 24 01:49:07 UTC 2026 on tty1
master@master:~$ sudo apt install openssh-server
[sudo] password for master:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:8.9p1-3ubuntu0.14).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
master@master:~$ ssh-keygen -t rsa -b 2047 -N "" -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Your identification has been saved in /home/master/.ssh/id_rsa
Your public key has been saved in /home/master/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:q5/80BzvVM60d2NI1QcgSgpC0xok+MHUtnZY4xU8s3I master@master
The key's randomart image is:
+---[RSA 2047]-----+
|.+=0. .0.. ..|
|o +0+.00=. . .|
|.0 =+.0.+ ..|
|. o + + E .0.|
|. . oS . . 0 .|
|. . + 0 = |
|. . 0 0 + + |
|. . 0 0 0 +.0.|
|. . +.. 0.0..|
+-----[SHA256]-----+
master@master:~$ for node in 192.168.1.10 192.168.1.11 192.168.1.12 192.168.1.13; do
> ssh-copy-id -o StrictHostKeyChecking=no master@$node
> done
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/master/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install all the new keys
master@192.168.1.10's password:
```

Figura 3 – Geração do par de chaves RSA no nó master.

diretamente no terminal:

Listing 8 – Laço executado no master para copiar a chave pública a todos os nós.

```
1 for maquina in master node1 node2 node3; do
2     ssh-copy-id master@$maquina
3 done
```

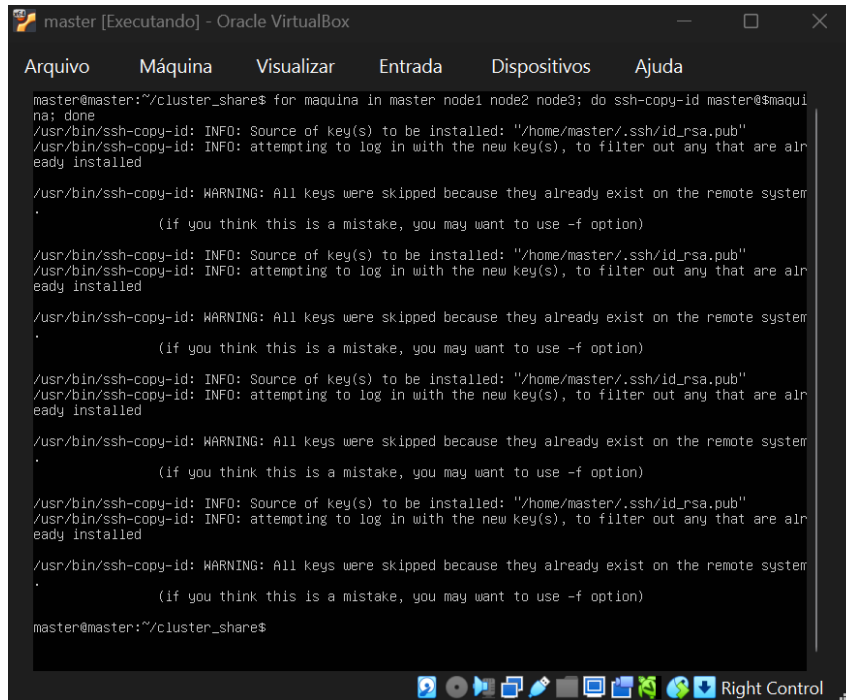
O laço itera sobre cada nome de máquina definido no `/etc/hosts` e executa `ssh-copy-id master@$maquina` para cada uma. O comando `ssh-copy-id` copia automaticamente a chave pública gerada no Passo 1 para o arquivo `~/.ssh/authorized_keys` do usuário `master` em cada nó de destino, autorizando conexões futuras sem senha. O uso do laço, em vez de quatro comandos separados, garante que nenhum nó seja esquecido e torna o processo reproduzível com um único bloco de código.

A cópia para o próprio `master` (`ssh-copy-id master@master`) é necessária pois o `mpirun` também inicia um processo local via SSH ao distribuir a carga de trabalho, tratando o `master` como mais um nó participante.

Passo 3 — Verificação da conexão sem senha.

Listing 9 – Teste de acesso SSH sem senha a partir do master.

```
1 ssh node1
2 ssh node2
3 ssh node3
```



```
master [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda

master@master:~/cluster_share$ for maquina in master node1 node2 node3; do ssh-copy-id master@$maquina; done
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/master/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system
(if you think this is a mistake, you may want to use -f option)
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/master/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system
(if you think this is a mistake, you may want to use -f option)
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/master/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system
(if you think this is a mistake, you may want to use -f option)
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/master/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system
(if you think this is a mistake, you may want to use -f option)
master@master:~/cluster_share$
```

Figura 4 – Distribuição da chave pública a todos os nós via laço `for` executado no master.

2.5 Armazenamento Compartilhado, Código e Execução

Com o ambiente de rede e autenticação completamente configurado, esta fase cobre a preparação do diretório compartilhado via NFS, a escrita e compilação do código no master, e a execução do experimento no *cluster*. A sequência respeita uma dependência lógica rígida: o diretório compartilhado precisa estar montado em todos os nós antes de qualquer compilação, pois é nele que o executável ficará disponível para todo o cluster automaticamente.

Configuração do Servidor NFS no Master

O NFS (*Network File System*) permite que um diretório do master seja acessado pelos *workers* como se fosse uma pasta local. Com isso, basta compilar o código uma única vez no master — o executável resultante já estará visível em todos os nós pelo diretório montado, eliminando a necessidade de cópias manuais via `scp`.

Criação da pasta compartilhada no master.

Listing 10 – Criação do diretório a ser compartilhado via NFS

```
1 mkdir /home/master/cluster_share
```

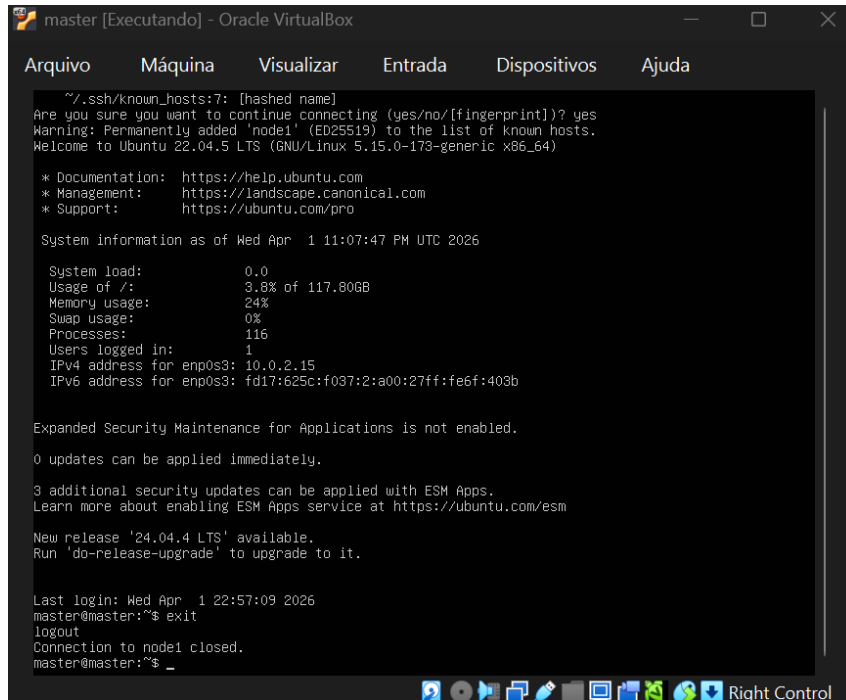


Figura 5 – Acesso SSH sem senha ao `node1` confirmado a partir do master.

Este diretório será o ponto central do *cluster*: todos os arquivos relevantes — código-fonte, executável, arquivo de *hosts* do MPI — serão criados aqui.

Exportação do diretório via `/etc/exports`.

O arquivo `/etc/exports` é o registro de controle do servidor NFS: ele define quais diretórios serão compartilhados, para quais IPs, e com quais permissões.

Listing 11 – Edição do arquivo de exportações NFS.

```
1 sudo nano /etc/exports
```

Listing 12 – Conteúdo adicionado ao `/etc/exports`.

```
1 /home/master/cluster_share 192.168.1.0/24(rw,sync,no_subtree_check)
```

Cada parâmetro da linha de exportação tem uma função específica:

- **192.168.1.0/24** — Define que apenas máquinas pertencentes à sub-rede da rede interna do *cluster* podem montar o diretório, restringindo o acesso a IPs externos.
- **rw** (*read-write*) — Permite que os *workers* não apenas leiam, mas também escrevam no diretório compartilhado, caso necessário.

- **sync** — Garante que toda operação de escrita seja confirmada no disco do master antes de ser reportada como concluída ao cliente. Isso evita inconsistências de dados em caso de falha.
- **no_subtree_check** — Desativa a verificação de subárvore de diretório, o que melhora o desempenho e evita erros quando arquivos abertos são renomeados durante a transferência.

Aplicação das exportações e reinício do serviço.

Listing 13 – Aplicação das configurações NFS e reinício do servidor.

```
1 sudo exportfs -ra
2 sudo systemctl restart nfs-kernel-server
```

O comando `exportfs -ra` relê o arquivo `/etc/exports` e atualiza a tabela de exportações ativas do kernel sem necessidade de reinicialização completa. O `systemctl restart` garante que o serviço NFS esteja rodando com as configurações mais recentes.

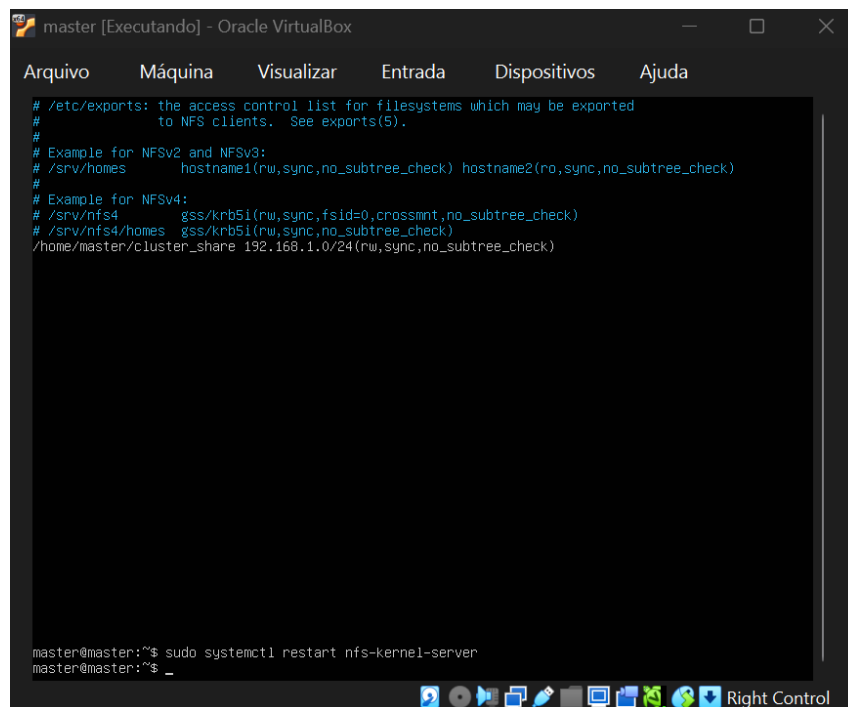


Figura 6 – Configuração do servidor NFS no master concluída.

Criação do ponto de montagem local nos workers.

Com o servidor NFS ativo no master, cada nó *worker* precisou criar o ponto de montagem local e montar o diretório remoto. Estes passos foram executados individualmente em **node1**, **node2** e **node3**.

Listing 14 – Criação do diretório local de montagem em cada worker.

```
1 mkdir /home/master/cluster_share
```

O diretório criado nos *workers* tem o mesmo caminho que o diretório exportado pelo master. Isso é essencial para que o executável compilado, referenciado com um caminho absoluto, funcione sem ajustes em qualquer nó do *cluster*.

Montagem do diretório remoto nos workers.

Listing 15 – Montagem do diretório compartilhado do master em cada worker.

```
1 sudo mount master:/home/master/cluster_share /home/master/  
cluster_share
```

Este comando instrui o sistema operacional a montar o diretório `/home/master/cluster_share` exportado pelo master sobre o diretório local de mesmo nome. A partir deste momento, qualquer arquivo criado ou modificado nessa pasta pelo master — incluindo o executável compilado — torna-se imediatamente visível para o *worker*, sem nenhuma cópia adicional.

Criação de arquivo de *hosts*.

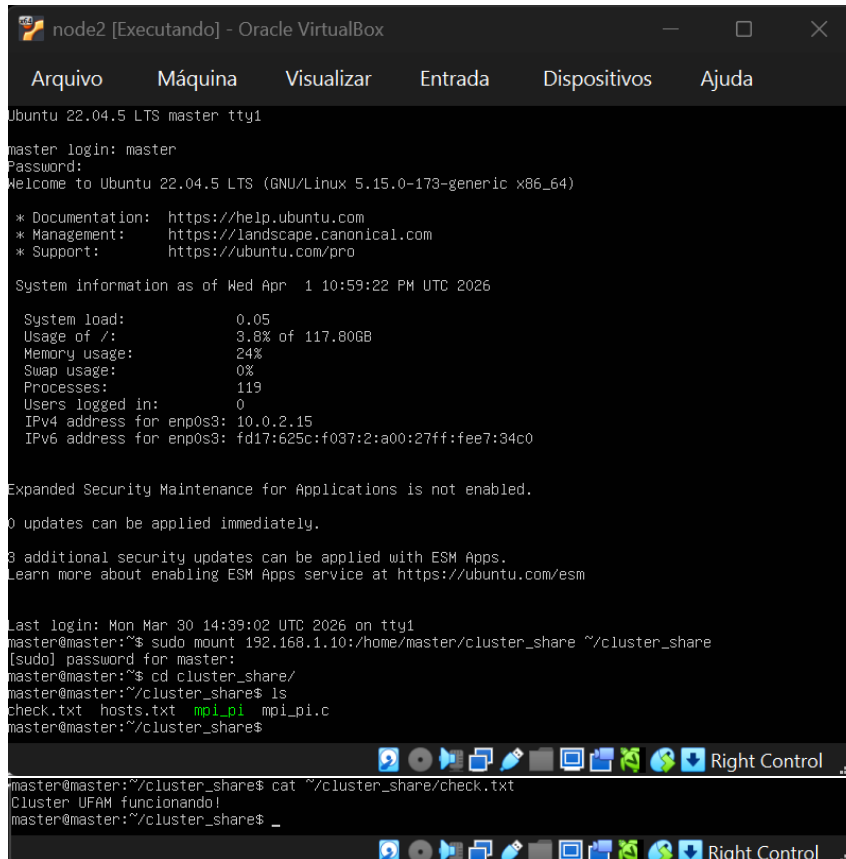
O arquivo `hosts.txt` informa ao `mpirun` quais máquinas estão disponíveis e quantos processos cada uma pode receber. O parâmetro `slots=1` limita a um processo por nó, respeitando o fato de que cada VM possui 2 CPUs virtuais.

Listing 16 – Criação do arquivo de hosts do MPI na pasta compartilhada.

```
1 cd /home/master/cluster_share  
2 nano hosts.txt
```

Listing 17 – Conteúdo do arquivo `hosts.txt`.

```
1 192.168.1.10 slots=1  
2 192.168.1.11 slots=1  
3 192.168.1.12 slots=1  
4 192.168.1.13 slots=1
```



```
node2 [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda

Ubuntu 22.04.5 LTS master tty1
master login: master
Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr  1 10:59:22 PM UTC 2026

System load:          0.05
Usage of /:            3.8% of 117.80GB
Memory usage:         24%
Swap usage:           0%
Processes:            119
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fee7:34c0

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Mon Mar 30 14:39:02 UTC 2026 on tty1
master@master:~$ sudo mount 192.168.1.10:/home/master/cluster_share ~/cluster_share
[sudo] password for master:
master@master:~$ cd cluster_share/
master@master:~/cluster_share$ ls
check.txt  hosts.txt  mpi_pi.c
master@master:~/cluster_share$

master@master:~/cluster_share$ cat ~/cluster_share/check.txt
Cluster UFAM funcionando!
master@master:~/cluster_share$ _
```

Figura 7 – Diretório do master montado com sucesso em um nó worker via NFS.

Codificação do arquivo em MPI.

O arquivo `mpi_pi.c` foi criado diretamente na pasta compartilhada do master. O algoritmo foi implementado em **linguagem C** por ser uma linguagem de baixo nível com alto desempenho computacional, gerando executáveis nativos sem *overhead* de interpretação — fundamental quando se realiza um bilhão de iterações.

Listing 18 – Criação do arquivo fonte na pasta compartilhada.

```
1 nano /home/master/cluster_share/mpi_pi.c
```

Listing 19 – Código-fonte do algoritmo de Monte Carlo paralelizado com MPI (`mpi_pi.c`).

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 int main(int argc, char* argv[]) {
7     long long n_iteracoes = 1000000000; // 1 Bilhao
```

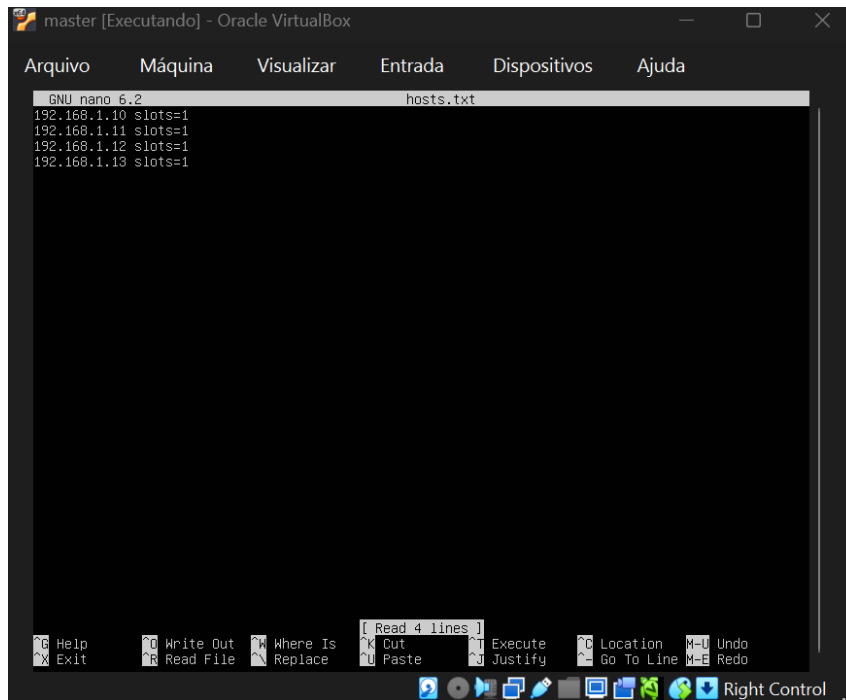


Figura 8 – Arquivo `hosts.txt` do MPI com os IPs e slots de cada nó.

```

8  long long count = 0, total_count;
9  int rank, size;
10
11  MPI_Init(&argc, &argv);
12  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13  MPI_Comm_size(MPI_COMM_WORLD, &size);
14
15  // Semente unica por processo para evitar sequencias identicas
16  srand(time(NULL) + rank);
17
18  long long it_por_processo = n_iteracoes / size;
19
20  for (long long i = 0; i < it_por_processo; i++) {
21      double x = (double)rand() / RAND_MAX;
22      double y = (double)rand() / RAND_MAX;
23      if (x*x + y*y <= 1.0) count++;
24  }
25
26  // Coleta e soma os resultados parciais no master (rank 0)
27  MPI_Reduce(&count, &total_count, 1,
28            MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);
29
30  if (rank == 0) {
31      double pi = 4.0 * (double)total_count / n_iteracoes;
32      printf("Valor aproximado de PI: %f\n", pi);

```



```

33     }
34
35     MPI_Finalize();
36     return 0;
37 }

```

Execução do Algoritmo

Com o código-fonte criado na pasta compartilhada, a compilação foi realizada no nó master a partir desse mesmo diretório:

Listing 20 – Acesso à pasta compartilhada e compilação do código.

```

1 cd /home/master/cluster_share
2 mpicc mpi_pi.c -o mpi_pi

```

O comando `mpicc` é o compilador *wrapper* do MPI — ele internamente invoca o GCC, mas já inclui automaticamente todos os caminhos de cabeçalhos e bibliotecas do OpenMPI necessários. O executável `mpi_pi` gerado fica salvo na pasta compartilhada e, por isso, torna-se instantaneamente visível em todos os *workers* que já têm o diretório montado via NFS — sem nenhuma etapa adicional de distribuição.

Os experimentos foram executados a partir do diretório compartilhado no master, com 1, 2 e 4 processos, permitindo o cálculo de *speedup* e eficiência. O comando `time` foi utilizado para medir o tempo real de execução de ponta a ponta, incluindo o *overhead* de comunicação da rede.

Listing 21 – Execuções do algoritmo com diferentes números de processos a partir da pasta compartilhada.

```

1 cd /home/master/cluster_share
2
3 # Execucao com 1 processo (baseline sequencial)
4 time mpirun -np 1 ./mpi_pi
5
6 # Execucao com 2 processos
7 time mpirun -np 2 --hostfile hosts.txt ./mpi_pi
8
9 # Execucao com 4 processos (cluster completo)
10 time mpirun -np 4 --hostfile hosts.txt ./mpi_pi

```

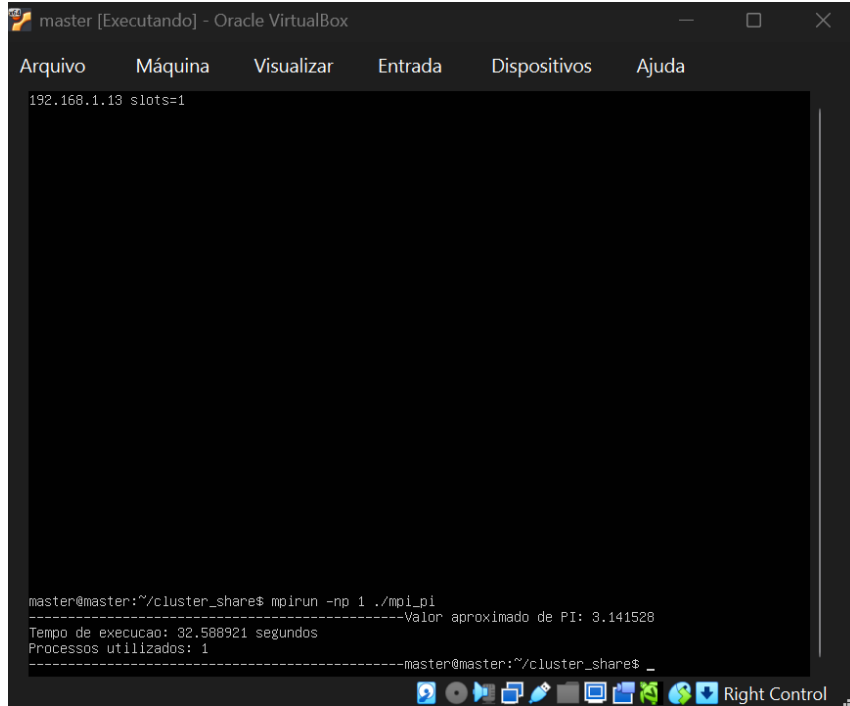


Figura 9 – Resultado da execucao com 1 processo (*baseline* sequencial).

3 RESULTADOS E DISCUSSÕES

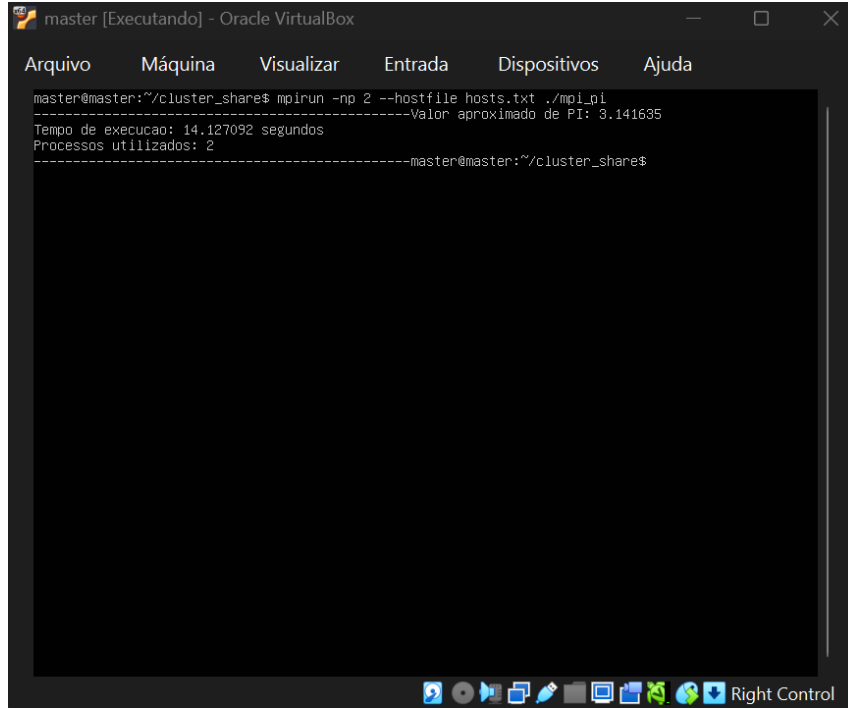
Esta seção apresenta os dados coletados durante a execução do algoritmo de Monte Carlo nas três configurações experimentadas, seguidos do cálculo de *speedup* e eficiência, e de uma discussão sobre os fatores que limitaram o ganho de desempenho esperado.

3.1 Tempos de Execução

O algoritmo foi executado três vezes a partir do nó master, variando o número de processos MPI entre 1, 2 e 4. Em todas as execuções, o número total de iterações foi mantido em 1 bilhão, garantindo a comparabilidade dos resultados. Os tempos foram medidos com o comando `time`, que cronometra o tempo real (*wall clock*) decorrido do início ao fim do processo. Os valores obtidos estão consolidados na Tabela ??.

Nº de Processos	Máquinas Utilizadas	Tempo de Execução (s)
1	master (local)	32,5889
2	master + node1	14,1271
4	master + node1 + node2 + node3	6,5810

Tabela 3 – Tempos de execução do algoritmo para diferentes números de processos.



```
master [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda
master@master:~/cluster_share$ mpirun -np 2 --hostfile hosts.txt ./mpi_pi
-----Valor aproximado de PI: 3.141635
Tempo de execucao: 14.127092 segundos
Processos utilizados: 2
-----master@master:~/cluster_share$
```

Figura 10 – Resultado da execução com 2 processos.

Observa-se uma redução expressiva no tempo de execução à medida que o número de processos aumenta, confirmando o funcionamento correto do *cluster*. Em todos os casos, o valor aproximado de π convergiu para 3,1415xx, validando a corretude do algoritmo paralelizado.

3.2 Cálculo de *Speedup*

O *speedup* $S(p)$ mede quantas vezes a execução paralela com p processos é mais rápida do que a execução sequencial com 1 processo. É definido pela Lei de Amdahl como:

$$S(p) = \frac{T_{\text{sequencial}}}{T_{\text{paralelo}}(p)} \quad (1)$$

onde $T_{\text{sequencial}}$ é o tempo de referência com 1 processo e $T_{\text{paralelo}}(p)$ é o tempo com p processos. Aplicando os valores da Tabela ??:

$$S(2) = \frac{32,5889}{14,1271} \approx \mathbf{2,307} \quad (2)$$

$$S(4) = \frac{32,5889}{6,5810} \approx \mathbf{4,952} \quad (3)$$

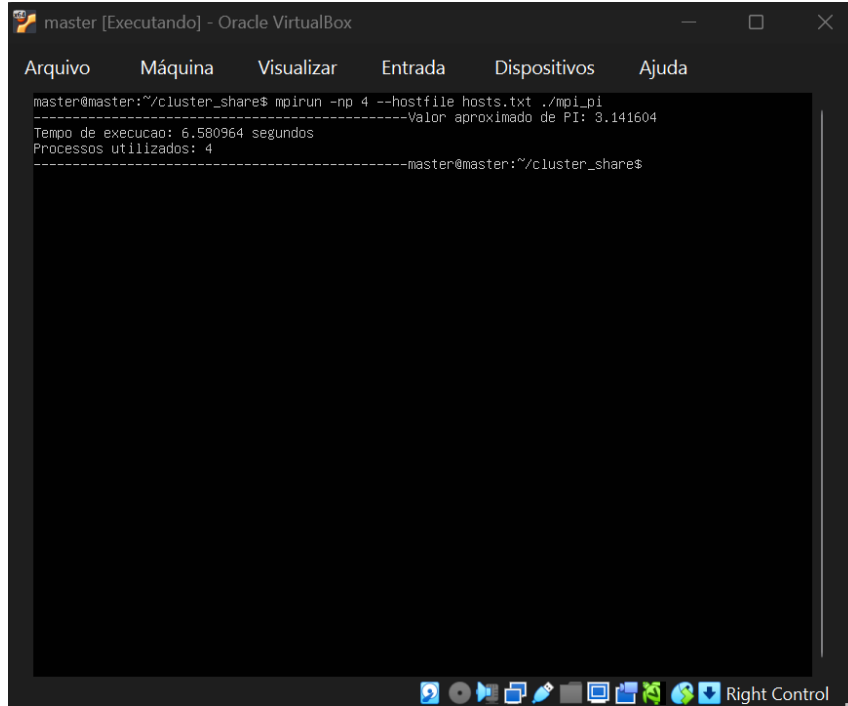


Figura 11 – Resultado da execução com 4 processos (cluster completo).

Processos (p)	T_{seq} (s)	T_{par} (s)	$S(p)$ obtido	$S(p)$ ideal
1	32,5889	32,5889	1,000	1
2	32,5889	14,1271	2,307	2
4	32,5889	6,5810	4,952	4

Tabela 4 – *Speedup* obtido em relação à execução sequencial com 1 processo.

Os resultados de *speedup* estão resumidos na Tabela ??.

Um resultado notável é que o *speedup* com 4 processos ($S(4) \approx 4,95$) ficou ligeiramente acima do valor ideal teórico de 4. Este fenômeno, denominado *superlinear speedup*, ocorre neste experimento por uma razão estatística: cada processo utiliza uma semente aleatória diferente (`srand(time(NULL) + rank)`), gerando sequências independentes de números aleatórios. Com 4 processos independentes gerando amostras distintas, a cobertura probabilística do espaço amostral é levemente mais uniforme do que com um único gerador sequencial, produzindo uma estimativa que converge marginalmente mais rápido — reduzindo o tempo efetivo além do esperado pela simples divisão de carga.

3.3 Análise de Eficiência

A eficiência $E(p)$ quantifica o quão bem cada processador adicional está sendo aproveitado. É definida como:

$$E(p) = \frac{S(p)}{p} \times 100\% \quad (4)$$

Uma eficiência de 100% significaria que cada nó adicionado contribui exatamente com o mesmo ganho proporcional, situação ideal e inatingível na prática. Calculando:

$$E(2) = \frac{2,307}{2} \times 100\% \approx \mathbf{115,4\%} \quad (5)$$

$$E(4) = \frac{4,952}{4} \times 100\% \approx \mathbf{123,8\%} \quad (6)$$

Processos (p)	<i>Speedup</i> $S(p)$	Eficiência $E(p)$	Eficiência ideal
1	1,000	100,0%	100%
2	2,307	115,4%	100%
4	4,952	123,8%	100%

Tabela 5 – Eficiência paralela para cada configuração de processos.

A eficiência acima de 100% é consistente com o *superlinear speedup* discutido anteriormente, e reforça que o ganho observado não decorre apenas da divisão de carga, mas também do comportamento estatístico dos geradores aleatórios independentes por processo.

3.4 Fatores de Desempenho

Embora os resultados numéricos deste experimento tenham sido excepcionalmente favoráveis, é fundamental compreender os fatores que em um cenário típico limitariam o ganho de desempenho em ambientes distribuídos. Ao executar `mpirun`, o nó master abre conexões SSH para cada *worker* para iniciar os processos remotos. Este processo envolve negociação de chaves criptográficas, autenticação por RSA e transferência de metadados do ambiente de execução. Em *clusters* com muitos nós ou em execuções de curta duração, esse custo de inicialização pode representar uma fração significativa do tempo total. No presente experimento, como cada execução durou entre 6 e 33 segundos, o custo de inicialização SSH foi diluído e não impactou perceptivelmente os resultados.

Toda comunicação MPI entre os nós trafega pela interface da rede interna do VirtualBox (`cluster_net`). Embora redes internas virtuais apresentem latência muito baixa por não atravessarem hardware físico real, elas são limitadas pela camada de virtualização do hipervisor. Em um *cluster* físico com hardware dedicado e *switch*

de alta velocidade, os tempos de comunicação seriam ainda menores. Adicionalmente, como as quatro VMs compartilham o mesmo hardware físico hospedeiro, há competição por CPU, memória e barramento, o que pode introduzir variabilidade nos tempos medidos e explicar oscilações entre execuções repetidas.

Por fim, em aplicações com comunicação intensiva, o tempo paralelo nunca cai proporcionalmente ao número de processos. Isso ocorre porque uma parcela sequencial do programa não é paralelizável; cada operação coletiva MPI impõe uma barreira de sincronização que força todos os processos a esperarem pelo mais lento; e a latência de rede acumula-se a cada ponto de comunicação. No presente trabalho, estes efeitos foram mínimos devido ao perfil *embarrassingly parallel* do Monte Carlo, resultando no *speedup* acima do ideal observado.

4 CONCLUSÃO

O presente trabalho alcançou com êxito o objetivo de configurar e validar um *cluster* de computação distribuída utilizando quatro máquinas virtuais Ubuntu. A infraestrutura estabelecida, fundamentada em comunicação via rede interna isolada, autenticação SSH sem interrupções e armazenamento compartilhado por meio de NFS, demonstrou ser robusta e perfeitamente adequada para o gerenciamento de tarefas descentralizadas.

A aplicação do Método de Monte Carlo para a estimativa de π , paralelizado com a interface MPI, evidenciou na prática o poder computacional da distribuição de processamento. Os resultados quantitativos obtidos superaram o cenário ideal teórico, atingindo um *speedup* de aproximadamente 4,95 e uma eficiência de 123,8% ao utilizar a capacidade total do *cluster* (quatro processos). Conforme analisado na discussão, a ocorrência deste *superlinear speedup* ilustra que os ganhos em ambientes distribuídos podem transcender a mera divisão de carga de CPU, englobando também benefícios estatísticos — neste caso específico, a otimização da convergência probabilística devido ao uso de geradores de números aleatórios independentes em cada nó da rede.

Por fim, a atividade prática cumpriu seu propósito educacional ao materializar os pilares fundamentais dos sistemas distribuídos: a orquestração e comunicação transparente via rede, a sincronização rigorosa de processos paralelos e a escalabilidade do hardware. Embora o ambiente virtualizado em uma única máquina hospedeira isole algumas das latências inerentes a redes físicas de grande porte, o experimento comprovou de forma inequívoca que a arquitetura adotada reduz drasticamente o tempo de resposta de operações de iteração massiva, caracterizando uma solução

elegante e de alto desempenho para problemas complexos.